PLAYWRIGHT POM FRAMEWORK WITH CLAUDE WEB AND CLI

Name	Tanny Rajbangshi
Project	SauceDemo Login Automation
Framework	Playwright Test
Programming Language	JavaScript
Tools	Node.js, VS Code, Playwright, Claude Web, JSON

1. Introduction

This assignment demonstrates how to build a basic Playwright automation framework using the Page Object Model (POM) design pattern. The SauceDemo website was selected as the application under test. Claude Web was used as an AI-assisted development tool to support code generation, while the framework was reviewed, adjusted, and executed manually.

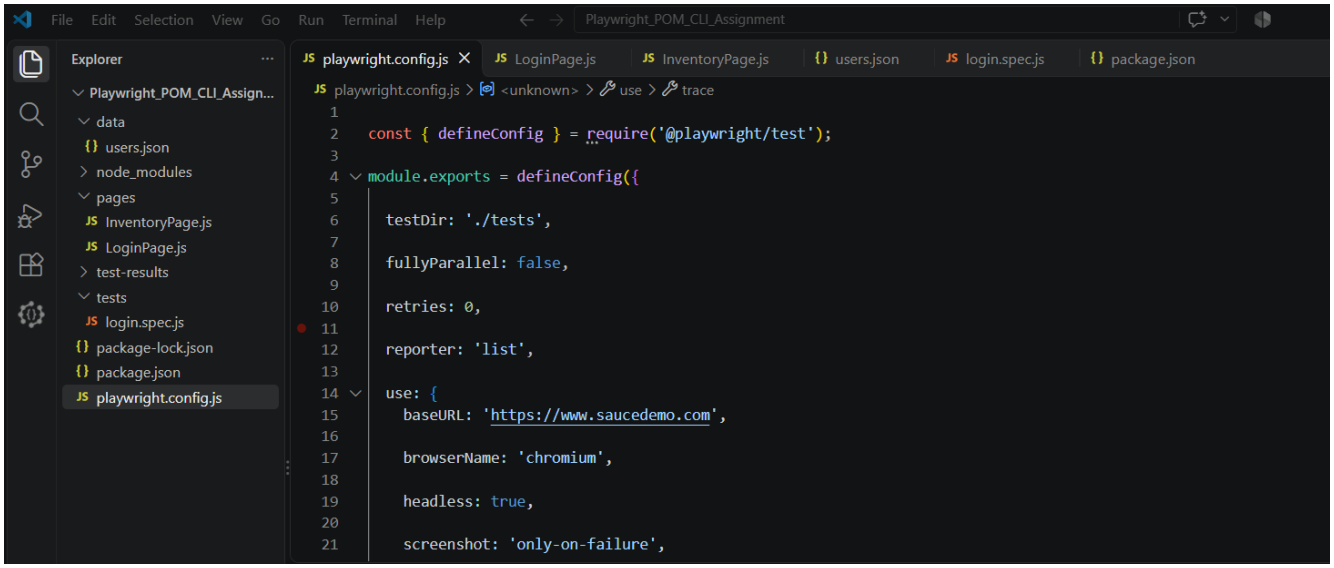
2. Objectives

- Create a reusable Playwright framework using the Page Object Model.
- Separate page locators and actions from test scripts.
- Implement login validation for valid and invalid user credentials.
- Use a JSON file to execute data-driven test cases.
- Run the automation tests through the command line using an npm script.
- Review and debug AI-generated code to ensure that it works correctly.

3. Project Structure

The project was organized into separate folders so that test scripts, page classes, and test data remain easy to maintain.

```
Playwright_POM_CLI_Assignment/
├── tests/
│   └── login.spec.js
├── pages/
│   ├── LoginPage.js
│   └── InventoryPage.js
├── data/
│   └── users.json
├── playwright.config.js
└── package.json
```



4. Page Object Model (POM) Implementation

The Page Object Model pattern was used to keep the framework modular and reusable. Each page class contains the locators and actions related to that page.

4.1 LoginPage.js

The LoginPage class stores the username field, password field, login button, and error-message locator. It provides a login() method to enter credentials and submit the login form, and a getErrorMessage() method to retrieve the displayed validation message.

4.2 InventoryPage.js

The InventoryPage class checks whether the inventory page has loaded successfully. It waits for the page title and product list to become visible, then returns the loading status. This helps verify that a successful login redirects the user to the inventory page.

5. Data-Driven Testing

Three test records were stored in data/users.json. The test script reads the JSON data and executes the same login workflow with different username and password combinations.

Test ID	Username	Password	Expected Result
TC-01	standard_user	secret_sauce	Successful login
TC-02	locked_out_user	secret_sauce	Error message displayed
TC-03	standard_user	wrong_password	Error message displayed

The positive test validates that a standard user can access the inventory page. The two negative tests validate that a locked-out user and an incorrect password are handled by the application with an error message.

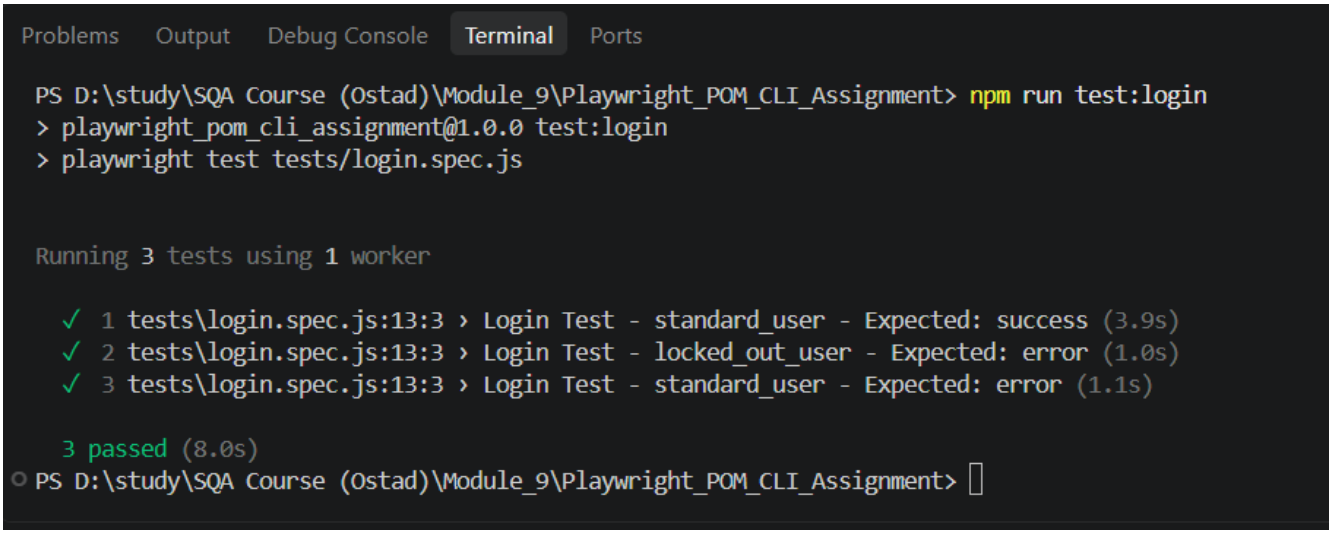
6. Test Execution and Results

The tests were executed from the project directory using the configured npm script:

npm run test:login

Total Tests	Passed	Failed	Status
3	**3**	**0**	**All Passed**

Execution result: all three data-driven login test cases passed successfully (3 passed, 0 failed). This confirms that the valid-login scenario and both invalid-login scenarios produced their expected results during the test run.



7. AI-Assisted Development and Manual Validation

Claude Web was used to assist with drafting the Playwright page classes, test scripts, and data-driven testing structure. The generated code was not accepted without review. Imports and exports, locator definitions, baseURL configuration, and page-load validation were checked and adjusted where necessary. The final code was then executed through the CLI to confirm the expected test results.

8. Conclusion

The assignment successfully demonstrates a basic Playwright POM framework with JSON-based data-driven testing. The framework separates page-level actions from test logic, making the code more readable, reusable, and maintainable. All three login test cases passed successfully, showing that the automation flow works for both positive and negative login scenarios.

— End of Report —